Bulletproofs 코드 흐름 상세 분석

개요

이 문서는 Bulletproofs 구현체의 코드를 단계별로 따라가면서 각 함수의 동작 과정과 데이터 흐름을 상세히 분석합니다. 실제 코드 실행 순서에 따라 설명하여 구현체의 동작 원리를 깊이 있게 이해할 수 있도록 합니다.

1. 범위 증명 생성 과정 (RangeProof::prove_single)

1.1 함수 호출 체인

```
RangeProof::prove_single()

↓

RangeProof::prove_single_with_rng()

↓

RangeProof::prove_multiple_with_rng()

↓

MPC 프로토콜 시뮬레이션
```

1.2 prove_single_with_rng 함수 분석

```
pub fn prove_single_with_rng<T: RngCore + CryptoRng>(

bp_gens: &BulletproofGens,
```

```
pc_gens: &PedersenGens,
transcript: &mut  Transcript,
v: u64, // 증명하려는 비밀 값
v_blinding: &Scalar, // 블라인딩 팩터
n: usize, // 비트 크기 (8, 16, 32, 64)
rng: &mut  T,
) -> Result<(RangeProof, CompressedRistretto), ProofError>
```

동작 과정:

1. **입력 검증**: 비트 크기가 유효한지 확인
2. **다중 값 함수 호출**: `prove_multiple_with_rng([v], [v_blinding], n, rng)` 호출
3. **결과 추출**: 첫 번째 커밋먼트만 반환

1.3 prove_multiple_with_rng 함수 상세 분석

이 함수는 실제 증명 생성의 핵심입니다.

1.3.1 초기 검증 단계

```
if values.len() != blindings.len() {
return Err(ProofError::WrongNumBlindingFactors);
}
```

- 값의 개수와 블라인딩 팩터 개수가 일치하는지 확인

1.3.2 MPC 프로토콜 설정

```
// 딜러 생성
let dealer = Dealer::new(bp_gens, pc_gens, transcript, n, values.len())?;


// 각 값에 대해 파티 생성
let parties: Vec<_> = values
.iter()
.zip(blindings.iter())
.map(|(&v, &v_blinding)| Party::new(bp_gens, pc_gens, v, v_blinding, n))
.collect::<Result<Vec<_>, _>>()?;
```

딜러 생성 과정 (Dealer::new):

1. **파라미터 검증**:

- 비트 크기가 8, 16, 32, 64 중 하나인지 확인
- 집계 크기가 2의 거듭제곱인지 확인
- 생성자 용량이 충분한지 확인

2. **트랜스크립트 초기화**:

```
let initial_transcript = transcript.clone();
transcript.rangeproof_domain_sep(n as u64, m as u64);
```

- 초기 트랜스크립트 상태 저장 (나중에 검증용)
- 도메인 분리자 추가

파티 생성 과정 (Party::new):

1. **파라미터 검증**: 비트 크기와 생성자 용량 확인

2. **값 커밋먼트 생성**:

```
let V = pc_gens.commit(v.into(), v_blinding).compress();
```

- Pedersen 커밋먼트: `V = v*B + v_blinding*B_blinding`

1.3.3 1단계: 비트 커밋먼트 생성

```
let (parties, bit_commitments): (Vec<_>, Vec<_>) = parties
.into_iter()
.enumerate()
.map(|(j, p)| {
p.assign_position_with_rng(j, rng)
.expect("We already checked the parameters, so this should never happen")
})
.unzip();
```

assign_position_with_rng 함수 분석:

1. **생성자 공유 획득**:

```
let bp_share = self.bp_gens.share(j);
```

2. **비트 분해**:

```
let mut a_L = Vec::with_capacity(n);
```

```
let mut a_R = Vec::with_capacity(n);

let mut exp_2 = Scalar::one();

for i in 0..n {

let a_L_i = Scalar::from((v >> i) & 1);

a_L.push(a_L_i);

a_R.push(a_L_i - Scalar::one());

exp_2 = exp_2 + exp_2;

}
```

- `a_L[i]` : i번째 비트 (0 또는 1)
- `a_R[i]` : `a_L[i] - 1` (-1 또는 0)
- 이는 `a_L[i] * a_R[i] = 0` 을 보장 (비트 제약)

3. **블라인딩 팩터 생성**:

```
let a_blinding = Scalar::random(rng);

let s_blinding = Scalar::random(rng);
```

4. **A 커밋먼트 계산**:

```
let mut A = self.pc_gens.B_blinding * a_blinding;

for (i, (a_L_i, a_R_i)) in a_L.iter().zip(a_R.iter()).enumerate() {

A += a_L_i * bp_share.G(n).nth(i).unwrap();

A += a_R_i * bp_share.H(n).nth(i).unwrap();

}
```

- `A = a_blinding*B_blinding + <a_L, G> + <a_R, H>`

5. **S 커밋먼트 계산**:

```
let mut S = self.pc_gens.B_blinding * s_blinding;

for (i, (s_L_i, s_R_i)) in s_L.iter().zip(s_R.iter()).enumerate() {

S += s_L_i * bp_share.G(n).nth(i).unwrap();

S += s_R_i * bp_share.H(n).nth(i).unwrap();

}
```

- `S = s_blinding*B_blinding + <s_L, G> + <s_R, H>`
- `s_L`, `s_R` 은 랜덤 블라인딩 벡터

1.3.4 2단계: 비트 챌린지 처리

```
let (dealer, bit_challenge) = dealer.receive_bit_commitments(bit_commitments)?;
```

receive_bit_commitments 함수 분석:

1. **커밋먼트 수 검증**:

```
if bit_commitments.len() != self.m {

return Err(MPCError::WrongNumBitCommitments);

}
```

2. **트랜스크립트에 커밋먼트 추가**:

```
for bit_commitment in bit_commitments.iter() {
```

```
transcript.append_point(b"V", &bit_commitment.V_j);

}

for bit_commitment in bit_commitments.iter() {

transcript.append_point(b"A", &bit_commitment.A_j.compress());

transcript.append_point(b"S", &bit_commitment.S_j.compress());

}
```

3. **챌린지 생성**:

```
let y = transcript.challenge_scalar(b"y");

let z = transcript.challenge_scalar(b"z");
```

- Fiat-Shamir 변환으로 랜덤 챌린지 생성

1.3.5 3단계: 다항식 커밋먼트 생성

```
let (parties, poly_commitments): (Vec<_>, Vec<_>) = parties

.into_iter()

.map(|p| p.apply_challenge_with_rng(&bit_challenge, rng))

.unzip();
```

apply_challenge_with_rng 함수 분석:

1. **벡터 다항식 구성**:

```
let y_inv = y.invert();

let exp_y = util::exp_iter(y);
```

```
let exp_2 = util::exp_iter(Scalar::from(2u64));

// l(x) = a_L - z*1^n + s_L*x

let mut l_poly = util::VecPoly1::zero(n);

for i in 0..n {

l_poly.0[i] = self.a_L[i] - z;

l_poly.1[i] = self.s_L[i];

}

// r(x) = y^n ⊙ (a_R + z*1^n + s_R*x) + z^2*2^n

let mut r_poly = util::VecPoly1::zero(n);

for (i, (exp_y_i, exp_2_i)) in exp_y.zip(exp_2).enumerate().take(n) {

r_poly.0[i] = exp_y_i * (self.a_R[i] + z) + (z_squared * exp_2_i);

r_poly.1[i] = exp_y_i * self.s_R[i];

}
```

2. **내적 다항식 계산**:

```
let t_poly = l_poly.inner_product(&r_poly);
```

- Karatsuba 방법으로 효율적 계산
- `t(x) = <l(x), r(x)> = t_0 + t_1*x + t_2*x^2`

3. **T_1, T_2 커밋먼트 생성**:

```
let t_1_blinding = Scalar::random(rng);

let t_2_blinding = Scalar::random(rng);

let T_1 = self.pc_gens.commit(t_poly.1, t_1_blinding);
```

```
let T_2 = self.pc_gens.commit(t_poly.2, t_2_blinding);
```

1.3.6 4단계: 다항식 챌린지 처리

```
let (dealer, poly_challenge) = dealer.receive_poly_commitments(poly_commitments)
```

receive_poly_commitments 함수 분석:

1. **T_1, T_2 커밋먼트를 트랜스크립트에 추가**
2. **x 챌린지 생성**:

```
let x = transcript.challenge_scalar(b"x");
```

1.3.7 5단계: 증명 공유 생성

```
let proof_shares: Vec<_> = parties
.into_iter()
.map(|p| p.apply_challenge(&poly_challenge))
.collect::<Result<Vec<_>, _>>()?;
```

apply_challenge 함수 분석:

1. **다항식 평가**:

```
let t_x = self.t_poly.eval(x);
let t_x_blinding = self.t_1_blinding * x + self.t_2_blinding * x * x;
```

2. **블라인딩 팩터 계산**:

```
let e_blinding = self.a_blinding + self.s_blinding * x;
```

3. **l, r 벡터 계산**:

```
let l_vec = self.l_poly.eval(x);
let r_vec = self.r_poly.eval(x);
```

1.3.8 6단계: 최종 증명 생성

```
let proof = dealer.receive_trusted_shares(&proof_shares)?;
```

receive_trusted_shares 함수 분석:

1. **증명 공유 집계**:

```
let t_x: Scalar = proof_shares.iter().map(|ps| ps.t_x).sum();
let t_x_blinding: Scalar = proof_shares.iter().map(|ps| ps.t_x_blinding).sum();
let e_blinding: Scalar = proof_shares.iter().map(|ps| ps.e_blinding).sum();
```

2. **l, r 벡터 연결**:

```
let mut l_vec = Vec::with_capacity(n * m);
let mut r_vec = Vec::with_capacity(n * m);
for ps in proof_shares.iter() {
l_vec.extend_from_slice(&ps.l_vec);
```

```
r_vec.extend_from_slice(&ps.r_vec);

}
```

3. **내적 증명 생성**:

```
let ipp_proof = inner_product_proof::InnerProductProof::create(

transcript,

&Q,

&G_factors,

&H_factors,

G_vec,

H_vec,

l_vec,

r_vec,

);
```

2. 내적 증명 생성 과정 (InnerProductProof::create)

2.1 함수 시그니처 분석

```
pub fn create(

transcript: &mut Transcript,

Q: &RistrettoPoint, // 내적을 위한 생성자

G_factors: &[Scalar], // G 생성자들의 스칼라 팩터
```

```
H_factors: &[Scalar], // H 생성자들의 스칼라 팩터

mut G_vec: Vec<RistrettoPoint>, // G 생성자 벡터

mut H_vec: Vec<RistrettoPoint>, // H 생성자 벡터

mut a_vec: Vec<Scalar>, // 왼쪽 벡터

mut b_vec: Vec<Scalar>, // 오른쪽 벡터

) -> InnerProductProof
```

2.2 초기화 단계

```
let mut G = &mut G_vec[..];

let mut H = &mut H_vec[..];

let mut a = &mut a_vec[..];

let mut b = &mut b_vec[..];


let mut n = G.len();


// 모든 벡터 길이가 같고 2의 거듭제곱인지 확인

assert_eq!(G.len(), n);

assert_eq!(H.len(), n);

assert_eq!(a.len(), n);

assert_eq!(b.len(), n);

assert!(n.is_power_of_two());
```

```
transcript.innerproduct_domain_sep(n as u64);
```

2.3 재귀적 압축 과정

2.3.1 첫 번째 반복 (최적화된 버전)

```
if n != 1 {
n = n / 2;
let (a_L, a_R) = a.split_at_mut(n);
let (b_L, b_R) = b.split_at_mut(n);
let (G_L, G_R) = G.split_at_mut(n);
let (H_L, H_R) = H.split_at_mut(n);

// 교차 내적 계산
let c_L = inner_product(&a_L, &b_R);
let c_R = inner_product(&a_R, &b_L);
```

2.3.2 L, R 포인트 생성

```
let L = RistrettoPoint::vartime_multiscalar_mul(
a_L.iter()
.zip(G_factors[n..2 * n].into_iter())
.map(|(a_L_i, g)| a_L_i * g)
.chain(
b_R.iter()
```

```
.zip(H_factors[0..n].into_iter())
.map(|(b_R_i, h)| b_R_i * h),
)
.chain(iter::once(c_L)),
G_R.iter().chain(H_L.iter()).chain(iter::once(Q)),
).compress();
```

- `L = <a_L, G_R'> + <b_R, H_L'> + c_L * Q`
- 여기서 `G_R' = G_R ⦿ g_factors`, `H_L' = H_L ⦿ h_factors`

2.3.3 챌린지 생성 및 벡터 압축

```
L_vec.push(L);
R_vec.push(R);

transcript.append_point(b"L", &L);
transcript.append_point(b"R", &R);

let u = transcript.challenge_scalar(b"u");
let u_inv = u.invert();

// 벡터 압축
for i in 0..n {
a_L[i] = a_L[i] * u + u_inv * a_R[i];
```

```
b_L[i] = b_L[i] * u_inv + u * b_R[i];

G_L[i] = RistrettoPoint::vartime_multiscalar_mul(

&[u_inv * G_factors[i], u * G_factors[n + i]],

&[G_L[i], G_R[i]],

);

H_L[i] = RistrettoPoint::vartime_multiscalar_mul(

&[u * H_factors[i], u_inv * H_factors[n + i]],

&[H_L[i], H_R[i]],

)

}
```

2.3.4 후속 반복들

```
while n != 1 {

n = n / 2;

// ... 유사한 과정 반복

// 단, G_factors, H_factors 최적화는 첫 번째 반복에서만

}
```

2.4 최종 결과 반환

```
InnerProductProof {

L_vec: L_vec,

R_vec: R_vec,
```

```
a: a[0], // 최종 압축된 스칼라

b: b[0], // 최종 압축된 스칼라

}
```

3. 범위 증명 검증 과정 (RangeProof::verify_multiple_with_rng)

3.1 초기 검증 단계

```
let m = value_commitments.len();


// 파라미터 검증

if !(n == 8 || n == 16 || n == 32 || n == 64) {

return Err(ProofError::InvalidBitsize);

}

if bp_gens.gens_capacity < n {

return Err(ProofError::InvalidGeneratorsLength);

}

if bp_gens.party_capacity < m {

return Err(ProofError::InvalidGeneratorsLength);

}
```

3.2 트랜스크립트 재생

```
transcript.rangeproof_domain_sep(n as u64, m as u64);


// 값 커밋먼트들 추가
for V in value_commitments.iter() {
transcript.append_point(b"V", V);
}


// 증명 구성 요소들 추가 및 검증
transcript.validate_and_append_point(b"A", &self.A)?;
transcript.validate_and_append_point(b"S", &self.S)?;


// 챌린지 재생성
let y = transcript.challenge_scalar(b"y");
let z = transcript.challenge_scalar(b"z");
```

3.3 내적 증명 검증 스칼라 계산

```
let (x_sq, x_inv_sq, s) = self.ipp_proof.verification_scalars(n * m, transcript)
```

verification_scalars 함수 분석:

1. **챌린지 재계산**:

```
let mut challenges = Vec::with_capacity(lg_n);
```

```
for (L, R) in self.L_vec.iter().zip(self.R_vec.iter()) {

transcript.validate_and_append_point(b"L", L)?;

transcript.validate_and_append_point(b"R", R)?;

challenges.push(transcript.challenge_scalar(b"u"));

}
```

2. **역원 계산**:

```
let mut challenges_inv = challenges.clone();

let allinv = Scalar::batch_invert(&mut challenges_inv);
```

3. **s 벡터 계산**:

```
let mut s = Vec::with_capacity(n);

s.push(allinv);

for i in 1..n {

let lg_i = (32 - 1 - (i as u32).leading_zeros()) as usize;

let k = 1 << lg_i;

let u_lg_i_sq = challenges_sq[(lg_n - 1) - lg_i];

s.push(s[i - k] * u_lg_i_sq);

}
```

3.4 메가 체크 계산

```
let mega_check = RistrettoPoint::optional_multiscalar_mul(
```

```
// 스칼라들의 이터레이터
iter::once(Scalar::one())
.chain(iter::once(x))
.chain(iter::once(c * x))
.chain(iter::once(c * x * x))
.chain(x_sq.iter().cloned())
.chain(x_inv_sq.iter().cloned())
.chain(iter::once(-self.e_blinding - c * self.t_x_blinding))
.chain(iter::once(basepoint_scalar))
.chain(g)
.chain(h)
.chain(value_commitment_scalars),
// 포인트들의 이터레이터
iter::once(self.A.decompress())
.chain(iter::once(self.S.decompress()))
.chain(iter::once(self.T_1.decompress()))
.chain(iter::once(self.T_2.decompress()))
.chain(self.ipp_proof.L_vec.iter().map(|L| L.decompress()))
.chain(self.ipp_proof.R_vec.iter().map(|R| R.decompress()))
.chain(iter::once(Some(pc_gens.B_blinding)))
.chain(iter::once(Some(pc_gens.B)))
.chain(bp_gens.G(n, m).map(|&x| Some(x)))
.chain(bp_gens.H(n, m).map(|&x| Some(x)))
```

```
.chain(value_commitments.iter().map(|V| V.decompress())),
)?;
```

3.5 최종 검증

```
if mega_check.is_identity() {
Ok(())
} else {
Err(ProofError::VerificationError)
}
```

4. 생성자 시스템 동작 과정

4.1 BulletproofGens 생성

```
pub fn new(gens_capacity: usize, party_capacity: usize) -> Self {
let mut gens = BulletproofGens {
gens_capacity: 0,
party_capacity,
G_vec: (0..party_capacity).map(|_| Vec::new()).collect(),
H_vec: (0..party_capacity).map(|_| Vec::new()).collect(),
};
gens.increase_capacity(gens_capacity);
```

```
gens

}
```

4.2 생성자 체인 생성

```
impl GeneratorsChain {

fn new(label: &[u8]) -> Self {

let mut shake = Shake256::default();

shake.input(b"GeneratorsChain");

shake.input(label);


GeneratorsChain {

reader: shake.xof_result(),

}

}

}


impl Iterator for GeneratorsChain {

type Item = RistrettoPoint;


fn next(&mut self) -> Option<Self::Item> {

let mut uniform_bytes = [0u8; 64];

self.reader.read(&mut uniform_bytes);
```

```
Some(RistrettoPoint::from_uniform_bytes(&uniform_bytes))

}

}
```

4.3 용량 증가 과정

```
pub fn increase_capacity(&mut self, new_capacity: usize) {

if self.gens_capacity >= new_capacity {

return;

}


for i in 0..self.party_capacity {

let party_index = i as u32;

let mut label = [b'G', 0, 0, 0, 0];

LittleEndian::write_u32(&mut label[1..5], party_index);

self.G_vec[i].extend(

&mut GeneratorsChain::new(&label)

.fast_forward(self.gens_capacity)

.take(new_capacity - self.gens_capacity),

);


label[0] = b'H';

self.H_vec[i].extend(
```

```
&mut GeneratorsChain::new(&label)
.fast_forward(self.gens_capacity)
.take(new_capacity - self.gens_capacity),
);
}
self.gens_capacity = new_capacity;
}
```

5. 트랜스크립트 프로토콜 동작

5.1 도메인 분리

```
fn rangeproof_domain_sep(&mut self, n: u64, m: u64) {
self.append_message(b"dom-sep", b"rangeproof v1");
self.append_u64(b"n", n);
self.append_u64(b"m", m);
}
```

5.2 챌린지 생성

```
fn challenge_scalar(&mut self, label: &'static [u8]) -> Scalar {
let mut buf = [0u8; 64];
self.challenge_bytes(label, &mut buf);
Scalar::from_bytes_mod_order_wide(&buf)
```

```
}
```

5.3 포인트 검증 및 추가

```
fn validate_and_append_point(
&mut self,
label: &'static [u8],
point: &CompressedRistretto,
) -> Result<(), ProofError> {
if point.is_identity() {
Err(ProofError::VerificationError)
} else {
Ok(self.append_message(label, point.as_bytes()))
}
}
```

6. 유틸리티 함수들의 역할

6.1 지수 이터레이터

```
pub fn exp_iter(x: Scalar) -> ScalarExp {
let next_exp_x = Scalar::one();
ScalarExp { x, next_exp_x }
```

```
}

impl Iterator for ScalarExp {
type Item = Scalar;
fn next(&mut self) -> Option<Scalar> {
let exp_x = self.next_exp_x;
self.next_exp_x *= self.x;
Some(exp_x)
}
}
```

6.2 거듭제곱의 합

```
pub fn sum_of_powers(x: &Scalar, n: usize) -> Scalar {
if !n.is_power_of_two() {
return sum_of_powers_slow(x, n);
}
if n == 0 || n == 1 {
return Scalar::from(n as u64);
}
let mut m = n;
let mut result = Scalar::one() + x;
let mut factor = *x;
```

```
while m > 2 {

factor = factor * factor;

result = result + factor * result;

m = m / 2;

}

result

}
```

6.3 벡터 다항식 내적

```
impl VecPoly1 {

pub fn inner_product(&self, rhs: &VecPoly1) -> Poly2 {

// Karatsuba 방법 사용

let l = self;

let r = rhs;


let t0 = inner_product(&l.0, &r.0);

let t2 = inner_product(&l.1, &r.1);


let l0_plus_l1 = add_vec(&l.0, &l.1);

let r0_plus_r1 = add_vec(&r.0, &r.1);


let t1 = inner_product(&l0_plus_l1, &r0_plus_r1) - t0 - t2;
```

```
Poly2(t0, t1, t2)
}
}
```

7. 메모리 관리 및 보안

7.1 민감한 데이터 소거

```
impl Drop for VecPoly1 {
fn drop(&mut self) {
for e in self.0.iter_mut() {
e.clear();
}
for e in self.1.iter_mut() {
e.clear();
}
}
}
```

7.2 상수 시간 연산

- 모든 스칼라 연산은 curve25519-dalek의 상수 시간 구현 사용
- 타이밍 공격 방지를 위한 설계

이 상세한 코드 흐름 분석을 통해 Bulletproofs 구현체의 각 단계별 동작 과정과 암호학적 원리를 깊이 있게 이해할 수 있습니다.